

PHP

Grundlagen, Dynamische Webseiten

Themenübersicht

Basics

- Grundlagen – dynamisches HTML
- Sprachkonstrukte
- Funktionen
- Closures
- Namespaces

Dynamische Webseiten

- Formulare
- Datenverarbeitung, Validierung, filter-Funktionen
- Datenbanken, Passwort-API
- Cookies & Sessions

- PHP 7
- Composer & PHP-Frameworks

Grundlagen – Dynamisches HTML

Entwickelt 1995 von Rasmus Lerdorf als „Personal Home Page“

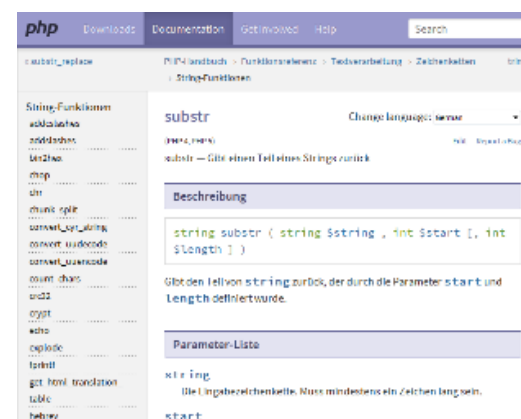
- Später umbenannt in „PHP: Hypertext Processor“
- Idee: **Im Web-Server integrierte Skriptsprache** zur Verarbeitung der Formular Daten der Browser, die mittels GET oder POST übertragen werden.

Historie

- Lange Zeit eine sehr simple Skriptsprache
- „Richtige“ Objektorientierung erst seit PHP 5.0
- PHP 6 als ursprünglicher Nachfolger von PHP 5 geplant
 - > Gescheitert auf Grund zahlreicher Probleme
 - > Versionsnummer 6 wurde nie vergeben
- Aktuelle Version: PHP 8.5 (2025)

PHP Documentation Group

- php.net



Grundlagen

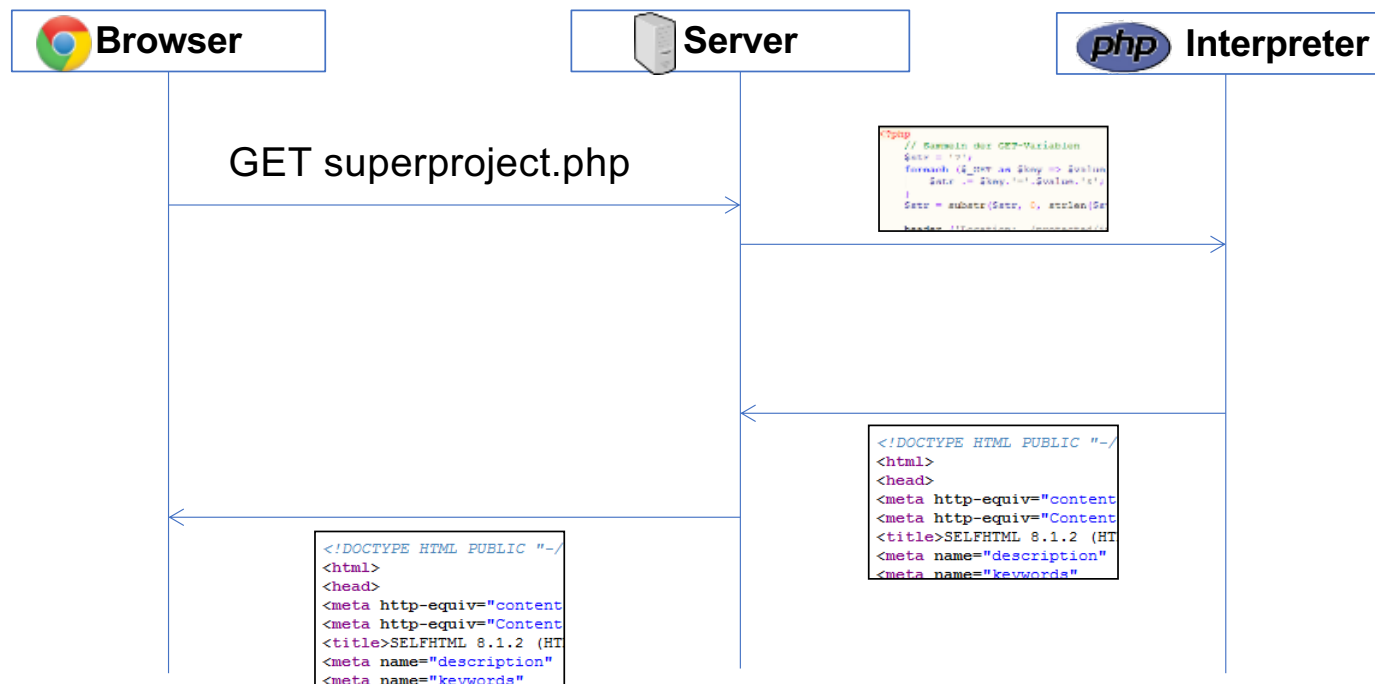
Voraussetzungen an den Server

- Installierter PHP-Interpreter notwendig
- Typisch für Webentwicklung ist ein „LAMP/WAMP“-Stack
 - > Linux/Windows, Apache, MySQL, PHP
 - > PHP ist dabei **in den Adressraum der Apache-Servers integriert**

Ablauf

- Browser stellt mittels GET (bei Formularen ggf. auch POST) die Anfrage an den Server
- **Dateiendung „.php“ signalisiert dem Server, dass die Datei PHP-Code beinhaltet**
- PHP-Interpreter durchläuft die Datei interpretiert den entsprechend markierten Quellcode und führt diesen direkt aus
- Die **Ausgabe** des ausgeführten Codes wird in das an den Browser gesendete HTML integriert, bzw. **ist das HTML-Dokument**
- Besonders sinnvoll ist der Einsatz von PHP im Zusammenhang mit Formulardaten

Aufruf einer PHP-Datei (vereinfacht)



Wichtig: Der Code ist für den Nutzer nicht sichtbar.
Es wird nur die Ausgabe der Ausführung übertragen

Beispiel: test.php

```
<?php  
    phpinfo();  
?>
```

PHP Version 5.3.10-1ubuntu3.6

System	Linux www.matse 3.2.0-45-generic #70-Ubuntu SMP Wed May 29 20:12:06 UTC 2013 x86_64
Build Date	Mar 11 2013 14:15:21
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php5/apache2
Loaded Configuration File	/etc/php5/apache2/php.ini
Scan this dir for additional .ini files	/etc/php5/apache2/conf.d
Additional .ini files parsed	/etc/php5/apache2/conf.d/curl.ini, /etc/php5/apache2/conf.d/gd.ini, /etc/php5/apache2/conf.d/imagick.ini, /etc/php5/apache2/conf.d/imap.ini, /etc/php5/apache2/conf.d/mcrypt.ini, /etc/php5/apache2/conf.d/mysqli.ini, /etc/php5/apache2/conf.d/mysql.ini, /etc/php5/apache2/conf.d/openssl.ini, /etc/php5/apache2/conf.d/sockets.ini, /etc/php5/apache2/conf.d/zip.ini

Syntax

Beispiel (ohne Formular-Verarbeitung)

beispiel2.php

```
<?php
    $titel = 'Web-Engineering';
    $gruss = 'Willkommen im Kurs!';
?>
<!DOCTYPE html>
<html>
<head>
    <title><?php echo $titel; ?></title>
</head>
<body>
    <?php echo $gruss; ?>
</body>
</html>
```

Hinweis: Wir werden Code von HTML über eine Template-Engine klarer trennen

beispiel2.php

- Produziert die folgende HTML-Ausgabe (ohne Code!!):

```
<!DOCTYPE html>
<html>
<head>
  <title>Web-Engineering</title>
</head>
<body>
  Willkommen im Kurs!
</body>
</html>
```


Vieles bekannt aus anderen Sprachen

- if, for, while...
- Funktionen
- Klassen und Objekte
- Semikolon beenden Anweisungen

Einige Konzepte unbekannt

- Fehlende Variablendeklaration
- Closures
- Cookies & Sessions

Variablen

- Dargestellt durch Dollar-Zeichen (\$) gefolgt vom Namen
- **Keine explizite Deklaration!**
- **Kein fester Typ!**
- Nicht initialisierte Variablen haben einen vom Kontext abhängigen Vorgabewert (false, null, leerer String oder leeres Array)

```
<?php
    $name = 'Max';
    $id = 42;
    $pi = 3.14;

    $foo += 5;    // 5!
?>
```



Datentypen

Typ	Beschreibung	Beispiel/Literal
boolean	Wahrheitswert	true, false
integer	Ganzzahl	0, 123, -12, 0xFF
float / double	Fließkommazahl (IEEE 754)	3.14, 1.2e5
string	Zeichenkette	'foo', "bar"
array	(assoziatives) Array	array(), array(1, 2, 3), array('key' => 'val')
object	Objekt	new Car (wobei Car eine Klasse ist)
NULL	Variable ohne Wert	NULL
resource	Referenz zu einer Ressource	mysql_connect('localhost', ...)
callable (nicht primär)	Funktionszeiger	function() { }

• Weitere Informationen: <http://php.net/manual/language.types.php>

Typumwandlung

- Implizit:

<code>\$foo = '5';</code>		<code>// '5'</code>
<code>\$foo += 3;</code>	<code>// ?</code>	<code>// 8</code>
<code>\$foo = '10 Mann' + \$foo;</code>	<code>// ?</code>	<code>// 18</code>
<code>\$foo += true;</code>	<code>// ?</code>	<code>// 19</code>
<code>\$foo += 1.5;</code>	<code>// ?</code>	<code>// 20.5</code>
<code>\$foo += array();</code>	<code>// ?</code>	<code>// FATAL ERROR</code>

- > Führt schnell zu Problemen
- > Besser explizit casten
 - Datenbanken erwarten oft einen bestimmten Typen
- > Implizit casten besser vermeiden!

Fatal error: Unsupported operand types in /lc/others/skriptprog/testing/test.php on line 9



Dies gilt es zu vermeiden, warum?

• Weitere Informationen: <https://php.net/manual/language.types.type-juggling.php>

Typumwandlung

- Explizit:

```
$foo = '1.5';           // '1.5'  
$foo = (double) $foo;   // 1.5  
$foo = (int) $foo;      // 1  
$foo = (boolean) $foo;  // true  
$foo = (string) $foo;   // '1'
```

cast to	false	1	0	1.7	-1.2	"	'1'	'php'	array()
boolean	false	true	false	true	true	false	true	true	false
int	0	1	0	1	-1	0	1	0	0
double	0	1	0	1.7	-1.2	0	1	0	0
string	"	'1'	'0'	'1.7'	'-1.2'	"	'1'	'php'	ERROR

Vergleich von Variablen

```
if (true == 'php' && 'php' == 0 && 0 != true) {  
    echo 'true == "php" == 0 != true';  
}
```



- > Mit == auf Gleichheit zu prüfen kann zu Problemen führen
 - Beispiel: [strpos](#)

\$a === \$b

- **Typstarker Vergleich!**
- Nur bei gleichem Typen und gleichem Inhalt von \$a und \$b wird true zurückgegeben

Typschwache Vergleiche mittels ==

	TRUE	FALSE	1	0	-1	"1"	"0"	"-1"	NULL	array()	"php"	""
TRUE	TRUE	FALSE	TRUE	FALSE	TRUE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	TRUE	FALSE	TRUE
1	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
0	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE
-1	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
"1"	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
"0"	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE
"-1"	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
NULL	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	TRUE
array()	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE
"php"	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE
""	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	TRUE

Quelle: <http://php.net/manual/de/types.comparisons.php>

Typstarke Vergleiche mittels ===

	TRUE	FALSE	1	0	-1	"1"	"0"	"-1"	NULL	array()	"php"	""
TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
1	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
0	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
-1	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
"1"	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
"0"	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE
"-1"	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
NULL	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
array()	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE
"php"	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE
""	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE

Quelle: <http://php.net/manual/de/types.comparisons.php>

Syntax

Strings (Unterscheidung einfache/doppelte Anführungszeichen)

Erzeugung eines Strings

- **Einfache** Anführungszeichen
- **Steuerzeichen** (\$, \n) werden **ignoriert** (bzw. nicht interpretiert)

- Beispiele:

```
echo 'Einfacher String';  
echo 'Auch ein Zeilenumbruch  
ist kein Problem, der wird aber beim Rendering im  
Browser aufgehoben';
```

```
echo 'I\'ll be back'; // I'll be back  
echo 'Dateien: C:\\*'; // Dateien: C:\*
```

```
$fuenf = 5;  
echo '2+3 = $fuenf'; // 2+3 = $fuenf
```

Da die Ausgabe der PHP-Anweisungen an den Browser gesendet wird, wir also zumeist dynamisches HTML erzeugen, kommt der String-Verarbeitung eine besondere Bedeutung zu

Erzeugung eines Strings

- **Doppelte** Anführungszeichen
- **Steuerzeichen** (\$, \n) werden **interpretiert**
- Variablen werden erkannt
- Beispiele:

```
echo "Zeilenumbruch folgt\n";
```

```
$fuenf = 5;
```

```
echo "2+3 = $fuenf";    // 2+3 = 5
```

```
echo "2+3 = {$fuenf}th"; // 2+3 = 5th
```

- Für Strings, die keine Variablen enthalten wird meist ein einfaches Anführungszeichen verwendet
- Für SQL-Queries wird meist ein doppeltes Anführungszeichen verwendet

Hinweis: Der oben genannte Zeilenumbruch verursacht bei der Darstellung im Browser keinen zwingenden Zeilenumbruch (Rendering). Hierzu muss der `<br>`-Tag verwendet werden

Verkettung

- Punktoperator (wichtig zur Erzeugung der Ausgabe)
- Beispiele:

```
$fh = 'FH' . ' ' . 'Aachen'; // 'FH Aachen'
```

```
$sql = "SELECT 'id' FROM 'users' "  
      . "WHERE 'name' = 'Max' ";
```

```
$id = 305;  
$ort = 'Raum ' . $id; // 'Raum 305'
```

```
$ort .= ' (3. OG)'; // 'Raum 305 (3. OG)'
```

- **Keine feste Größe**
- **Assoziative Arrays** möglich (eigentlich sogar der **Standard**)
- Es gibt numerische Arrays, assoziative (nur Strings als Key) und mehrdimensionale
- Mischformen von **String** und **Integern als Key** erlaubt
 - > Faktisch als numerische und assoziative gemischt
 - > Bedeutet nicht, dass dies auch sinnvoll ist...
- Als Wert ist alles erlaubt!
- Numerische Arrays starten mit 0, können aber Lücken haben
 - > Vorsicht! PHP gibt keine Warnungen aus wenn Sie auf nicht definierte Array-Indizes zugreifen. Besser mit `isset()` oder `array_key_exists()` prüfen
 - > Wert ist dann null

Erstellung:

```
$normal = [];  
$mix1   = [ 2, 4, 8, 'Tomate' ];  
$mix2   = [ 'foo' => 'bar', 12 => true ];  
$colors = ['rgb1' => [0,0,255)];
```

Zugriff und Bearbeitung:

```
$arr = []           // Alt: array()
$arr[] = 'EINS';    // array('EINS')
$arr[] = 'ZWEI';    // array('EINS', 'ZWEI')

$arr[0] = 'NEU';    // array('NEU', 'ZWEI')
$arr['a'] = 5;       // array(0 => 'NEU', 1 => 'ZWEI',
                    //           'a' => 5)

echo $arr[1];        // 'ZWEI'
```

Grundsätzliches bekannt aus Java

- Bedingungen

- > if
- > else if
- > else

- Schleifen

- > for
- > while
- > do-while
- > break
- > continue

foreach-Schleife

```
foreach ($arr as $value) {  
    // ...  
}
```

```
foreach ($arr as $key => $value) {  
    // ...  
}
```

Weitere Informationen: <http://php.net/manual/control-structures.foreach.php>

Dateien mittels *require* und *include* einbinden

- Ermöglicht Auslagerung und Mehrfachnutzung des Codes
- Bindet den Code aus der Datei ein
- Geltungsbereich von der Position des Befehlsaufruf abhängig

```
require './lib/Database.class.php'
```

- > „Fatal Error“, falls ein Fehler beim Einbinden auftritt
 - Die Ausführung wird sofort abgebrochen
 - Was aber ist mit der eigentlich anzuzeigenden Web-Seite?

```
include './lib/Encoding.class.php'
```

- > Warning, falls ein Fehler beim Einbinden auftritt
 - Skript wird weiterausgeführt
 - Web-Seite kann angezeigt werden

Dateien mittels *require* und *include* einbinden

- *include* nur bei Dateien, die **für den Programmablauf optional** sind
 - > I.d.R. ist die Benutzung von *require* sinnvoller, aber bitte ohne Fehler!
- „Geklammerte Syntax“ möglich

```
require ('./lib/Database.class.php') ;  
include ('./lib/Encoding.class.php') ;
```

- Eine (relative oder absolute) Pfadangabe führt dazu, dass nicht im *include_path* gesucht wird
 - > *include_path* gibt an in welchem Verzeichnis PHP einzubindende Dateien suchen soll (für Bibliotheken zu gebrauchen)
 - > php.ini setzt diesen systemweit, *set_include_path()* zur Laufzeit
 - > **Unbedingt vermeiden, dass hier Benutzereingaben greifen können**
- *require_once* und *include_once* binden die Datei nur einmal ein, auch wenn der Funktionsaufruf öfter erfolgt

```
require_once ('./lib/Database.class.php') ;
```

Sonstige Befehle

`bool isset($variable)`

- > Prüft ob eine Variable existiert und nicht *NULL* ist

`bool empty($var)`

- > Prüft, ob eine Variable einen Wert enthält
- > Für die folgenden Werte wird *TRUE* zurückgegeben:
 - Leere Zeichenkette (*"*), leeres Array (*array()*)
 - *0*, *0.0*, *'0.0'*
 - *NULL*, *FALSE*
 - In einer Klasse deklarierte, aber nicht belegt Variable

Sonstige Befehle

```
bool define($name, $value)
```

- > Setzt Konstante die in allen Geltungsbereichen verfügbar ist
- > Nur für Werte benutzen, die für die ganze Applikation relevant sind

- > Beispiele:

```
define('ROOT_DIR', '/my_project');  
define('VERSION', '3.1.24');
```

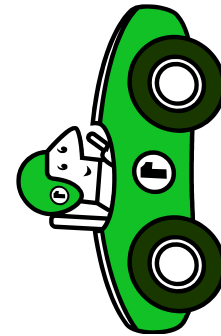
- > Auslesen des Wertes **ohne \$-Zeichen**:

```
echo VERSION;
```

Syntax

Objektorientierung - Klassen

```
class Auto {  
    private $farbe = 'blau';  
    private $ps;  
    public static $speedLimit = 100;  
  
    public function __construct($ps = 60) {  
        $this->ps = $ps;  
    }  
    public function getPs() {  
        return $this->ps;  
    }  
}  
  
$auto = new Auto();           // Instanz der Klasse Auto  
$geschw = $auto->getPs(); // 60
```



Klassengestützte OOP

- Zugriff auf Membervariablen

```
$this->name    // Innerhalb der Klasse
$objekt->name   // Außerhalb der Klasse (falls public)
self::name     // Zugriff auf statische Variable
parent::name   // Zugriff auf Elternklasse (statisch)
```

```
class Beispiel {
    public static $statischeVariable = 10;
}
echo Beispiel::$statischeVariable; // Ausgabe: 10
```

- Konstruktor

```
function __construct() { ... }
```

- Destruktor

```
function __destruct() { ... }
```

Wie Variablen sind auch Funktionen nicht typsicher!

- Funktionen können manchmal Zahlen, manchmal Strings zurückgeben
- In der Regel ist es aber sinnvoll, dass eine Funktion nur einen Typen zurückgibt

Angabe von Vorgabewerten möglich

- Beispiel:

```
function machKaffee($typ = 'Kaffee') {  
    return 'Eine Tasse ' . $typ;  
}
```



```
echo machKaffee(); // 'Eine Tasse Kaffee'  
echo machKaffee('Cappuccino'); // 'Eine Tasse Cappuccino'  
echo machKaffee('Espresso'); // 'Eine Tasse Espresso'
```

Call-By-Reference

```
function toggle(&$value) {  
    $value = ($value + 1) % 2;  
}  
$a = 1;  
toggle($a);  
// $a === 0
```

Variable Anzahl von Parametern

```
function tolerant() {  
    $numargs = func_num_args(); // Anzahl  
    $arg1 = func_get_arg(0);    // 1. Parameter  
    $args = func_get_args();    // Array mit Parametern  
}  
tolerant(1, 2, 4);
```


Syntax

Funktionen - Geltungsbereich

Der **Geltungsbereich von Variablen ist durch Funktionen gegliedert.**

Normalerweise sind Variablen damit nur innerhalb der Funktion bekannt, in der sie „deklariert“ wurden und damit weder in den äußeren noch in den inneren Bereichen.

Variablen im Geltungsbereich außerhalb der Funktion sind nur durch „global“-Schlüsselwort zu erreichen

▪ Beispiel:

```
$a = 1;
$b = 2; // $b hat hier einen local scope
function incA() {
    global $a;    // $b ist unbekannt,
                  // $a hat globalen scope
                  // global muss vor der Verwendung deklariert werden

    $a++;
}
incA();
echo $a; // 2
```

Lambdas

- Anonyme Funktionen ohne Namen
- Idee: **Funktionszeiger** wird in Variable gespeichert
 - > Variable kann wie Funktion benutzt werden
 - > Kann als Argument übergeben werden
- Generische Programmierung durch Parametrisierung der aufgerufenen Funktion möglich (z.B. durch Übergabe der Funktion als Parameter)



Lambdas

```
$eineFunktion = function () {  
    echo 'Ich bin eine anonyme Funktion!';  
};
```

```
// Aufruf:  
$eineFunktion();
```

Lambdas

- Es geht auch mit Parametern

```
$array = range(1, 10);           // Array mit [1,...,10]

$mySort = function($a, $b) { // Anonyme Funktion
    return $b - $a;           // gespeichert in $mySort
};

usort($array, $mySort);         // Eigene Funktion
                                // zur Sortierung nutzen
```

Weitere Informationen: <https://php.net/manual/en/function.usort.php>

Syntax

Verwendung von Funktionen als Parameter

Lambdas

```
function echoText(callable $callback) {  
    echo $callback();  
}
```

Ein callable ist eine Funktion, die aufgerufen werden kann

```
echoText(function () {  
    return 'Ich bin cool';  
});
```

Wir sehen hier das Prinzip einer Wegwerffunktion

```
echoText(function () {  
    return 'Ich bin cooler';  
});
```

```
$eineFunktion = function () {  
    return 'Ich der coolste!';  
};  
echoText($eineFunktion);
```

Closures

```
$dayPrepare = function() {  
    $dayNames = array('Mo', 'Di', 'Mi', 'Do',  
                      'Fr', 'Sa', 'So');  
  
    return function($day) use ($dayNames) {  
        return $dayNames[$day];  
    };  
};
```

Anonyme Funktion in einer anonymen Funktion

Variable \$dayNames aus äußerem Geltungsbereich kann mittels use im inneren verwendet werden!

```
$dayNameGetter = $dayPrepare();  
  
echo $dayNameGetter(1); // Di
```

Aufruf der inneren anonymen Funktion, die aufgrund der use-
"Brücke" immer noch Zugriff auf die Variable **\$dayNames** hat.
Wir gehen hier von Lambdas über zu Closures (im nicht puristischen Sinn)

Closures

- Ein Lambda wird zum **Closure**, wenn es **Zugang zu Variablen** bekommt, die **normalerweise nicht zur Verfügung stehen** würden
- In PHP können Closures sowohl als anonyme als auch als benannte Funktionen auftreten. Leider können viele Programmiersprachen nicht beides, so dass unglücklicherweise Closures oft mit anonymen Funktionen/Lambdas gleichgesetzt werden
- In PHP werden Variablen in Closures nicht automatisch erfasst, sondern müssen mit **use** explizit gebunden werden

```
function func() {  
    $i = 0;  
}  
// $i ist hier nicht definiert  
isset($i); // false
```

```
function func() {  
    $i = 1;  
    return function() use ($i) {  
        echo $i;  
    };  
}  
  
$x = func();  
$x(); // 1
```

use erfasst (captured)
standardmäßig by value

Syntax

Closures vs static

Closures

- Closures erlauben im Gegensatz zu static-Variablen eine beliebig häufige Instanziierung
- Closures ermöglichen das Prinzip der Dependency Injection

```
function counter_static() {  
    static $c = 0;  
    return ++$c;  
}  
  
echo counter_static(); // 1  
echo counter_static(); // 2
```

```
$counter = (function () {  
    $c = 0;  
    return function () use (&$c) {  
        return ++$c; };  
})();  
  
echo $counter(); // 1  
echo $counter(); // 2
```

use captured \$c nun
by reference



Closure-Fabrik:

```
function makeCounter() { $c = 0; return function() use (&$c) { return ++$c; }; }  
$cnt = makeCounter();  
echo $cnt(); // 1  
echo $cnt(); // 2
```


Syntax

Closures vs static

Closure-Fabrik:

```
function makeCounter() { $c = 0; return function() use (&$c) { return ++$c; }; }
$cnt = makeCounter();
echo $cnt(); // 1
echo $cnt(); // 2
```

Closure-Fabrik mit Dependency Injection:

```
class Counter {
    private int $c = 0;
    public function inc(): int { return ++$this->c; }
    public function get(): int { return $this->c; }
}
function makeCounterClosure(Counter $counter): callable {
    return function() use ($counter) { return $counter->inc(); }; }

$counter = new Counter();
$fn=makeCounterClosure($counter);
echo $fn(); // 1
echo $fn(); // 2
echo $counter->get(); // 2
```

Exceptions (wie bekannt)

```
try {  
    funcWhichMightThrowException();  
} catch (Exception $e) {  
    echo 'Problem calling ...: ' . $e->getMessage();  
}
```

- Eigene Exception-Klassen können von Exception abgeleitet werden

```
class MyException extends Exception {...}  
throw new MyException();
```

- Beim Abfangen kann der Exception-Name angegeben werden
- Die meisten PHP-Funktionen werfen im Fehlerfall leider keine Exceptions

Weitere Informationen: <http://php.net/manual/language.exceptions.php>

Unterschied: Errors & Exceptions

- Errors werden von PHP bei Fehlersituationen geworfen (und führen normalerweise zum Abbruch des Skripts).
- Aber auch Errors können gefangen werden:

```
errorHandler($errLevel, $errMsg, $errFile, $errLine) {  
    throw new ErrorException($errMsg, $errLevel, $errLevel,  
        $errFile, $errLine);  
}  
set_error_handler("errorHandler");
```

- Fatal-Errors (nicht alle!) können oftmals abgefangen werden (**ein Skript sollte niemals mit einem Fatal-Error abbrechen**):

```
register_shutdown_function("fatal_handler");
```

Weitere Informationen: <https://www.sitepoint.com/error-handling-in-php/>